

Where the Work **Runs.**

The systems, stores, and pipelines one constraint depends on — and whether AI can actually reach them.

You can't design a system you can't see.
Knowledge That Walks mapped the human inputs. This maps the digital ones. Together they're the picture Design needs.

List the **sources**.

Every task in the constraint workflow consumes information from somewhere and sends it somewhere else. List every digital source that touches it, then mark which **layer** it lives in. The layer tells Design what exists to work with — and what has to be built before any agent can reason reliably.

- SYSTEM OF RECORD**
- Authoritative data.** CRM, ERP, QuickBooks, the job-costing database. If it's wrong here, it's wrong everywhere.
- SYSTEM OF KNOWLEDGE**
- Institutional know-how.** SOPs, playbooks, pricing policy, the doc that explains how exceptions actually work.
- SYSTEM OF SEMANTICS**
- Meaning & relationships.** What ties "Client A" in the CRM to "Account 0042" in the ERP. Most companies don't have one yet — note it as a gap.

1.

One row per source: shared drives, inboxes, spreadsheets, the CRM, the ERP, the process logs.
2.

Tag the **layer** and name a real **owner**. "Nobody" is a finding, not a blank.
3.

Mark **status**: Clean (usable as-is), Needs Work (quality/format problems), or Missing (should exist, doesn't).

SOURCE	LAYER	OWNER	STATUS
<i>e.g. HubSpot CRM — win/loss, deal sizes</i>	<i>Record</i>	<i>Ty (Sales)</i>	<i>Needs Work</i>
<i>e.g. JobBOSS ERP — job costing history</i>	<i>Record</i>	<i>Finance</i>	<i>Clean</i>
<i>e.g. Customer Notes.xlsx — pricing exceptions</i>	<i>Knowledge</i>	<i>Elena</i>	<i>Needs Work</i>

WATCH FOR — THE DATA-AUDIT TRAP

If every row is a **System of Record** and none is Knowledge or Semantics, you ran a data audit, not Source. The know-how that makes the constraint hard usually lives in the Knowledge layer — or in nobody's system at all. Keep it scoped to the one constraint; if it doesn't touch the workflow, it doesn't go on the map.

Run the pipeline audit.

Knowing where data lives isn't enough — Design needs to know whether it **moves**. For each source, ask: does it connect to the other systems around the constraint, or is a person carrying data by hand? Mark each one **Connected**, **Manual**, or **Broken**.

- API****How systems talk to systems.** Your CRM has one; your ERP has one. Other software can ask it questions and get structured answers back.
- MCP****How AI talks to your systems.** Lets a model read your CRM or check a status live — the difference between a chatbot and a workflow.
- CONNECTOR****Pre-built integration.** Zapier, Make, n8n — links two systems without custom code. The thing companies buy *before* doing this work.

1. **Connected** — an API, MCP, or connector already carries data between this source and the systems around it.

2. **Manual** — a person is carrying data a connection could carry (export, re-key, copy-paste).

3. **Broken** — the data doesn't move at all; it sits in one place nobody can reach programmatically. Every Manual or Broken row is your handoff to Design.

SOURCE	CONNECTED / MANUAL / BROKEN	LAST VERIFIED ACCURATE?
<i>e.g. JobBOSS ERP</i>	<i>Manual — Elena queries by memory, not sys</i>	<i>Unknown</i>
<i>e.g. Customer Notes.xlsx</i>	<i>Broken — desktop file, no backup, no sync</i>	<i>Never validated</i>

PRO TIP

You don't need to understand the engineering behind APIs or MCPs. You need to know whether your systems talk to each other **or a person is carrying data between them**. That's the whole question this column answers — and it's what you hand a contractor or leave for Design to wire.

Tier it for AI.

Each source is reachable by AI today, reachable with extraction work, or not reachable at all. The tier tells Design what to use directly, what to build a pipeline for, and what stays human. Run it while the map is fresh — it goes fast.

- Tier 1 – use directly • structured & API-accessible
- Tier 2 – extract first • unstructured but capturable
- Tier 3 – human judgment • not AI-accessible yet
- Not AI – ephemeral / in-the-moment

1.

Tag each source **1, 2, 3**, or **Not AI**. Also note structured vs. unstructured.
2.

An AI workflow that mixes Tier 1 truth with a stale Tier 3 archive will **confidently cite your 2019 pricing**. The tiers exist to prevent exactly that.
3.

A valuable Tier 2 source (a transcript, a PDF of SOPs) is a Design decision: how much structure to add, and where.

SOURCE	TIER	STRUCTURED?	WHAT AI CAN DO WITH IT
<i>e.g. CRM / ERP records</i>	<i>1</i>	<i>Yes</i>	<i>Query in real time</i>
<i>e.g. Documented SOPs, transcripts, PDFs</i>	<i>2</i>	<i>No</i>	<i>Extract, then index</i>
<i>e.g. Undocumented relationship context</i>	<i>3</i>	<i>—</i>	<i>Nothing yet — keep h</i>

WHY THIS MATTERS DOWNSTREAM

Agents treat every accessible piece of information as equally valid unless you tell them otherwise. A human handed bad data slows down and asks a question. An agent handed bad data **executes against it, at scale, every time**. The tier and the status are how you stop that before Build.

Name what's missing.

The first three steps describe what exists. This column reveals the most risk: knowledge a designer would need that **isn't represented anywhere** — questions your current sources can't answer, decisions made by gut today that should be made by data. Every missing source is a potential Sprint failure mode. Name them before Build, not after deployment finds them.

1.

List every question a designer would ask that your current sources **can't** answer.
2.

For each, note what it **should** be — a digital system, a documented process, captured expertise — and what it'd take to create.
3.

If the Missing column is empty, you didn't look hard enough. Zero gaps is a red flag, not a clean bill.

WHAT'S MISSING — THE QUESTION WE CAN'T ANSWER	SHOULD BE	WHAT IT'D TAKE
<i>e.g. Are our fast estimates actually accurate vs. final cost?</i>	<i>Joined data</i>	<i>Compare ERP actuals to quoted</i>
<i>e.g. Which customer segments are most profitable to quote</i>	<i>Joined data</i>	<i>Link CRM win/loss to ERP job co</i>

WORKED EXAMPLE — MERIDIAN'S TWO MISSING INPUTS

Two things that didn't exist yet — **historical quote-to-actual accuracy** and **customer-segment profitability** — were the exact inputs the team needed to decide whether an AI-assisted quoting workflow was even viable. **Missing** The data to build them sat in the ERP and the CRM. Nobody had ever joined them. None of this surfaces in a general data audit — only by working it against one constraint.

Assemble the **map**.

Now put it on one page. Pull your digital rows together with the human sources from **Knowledge That Walks** into a single six-column Knowledge Map — the input Design needs, and all it needs. Then run the check.

SOURCE	TYPE	OWNER	STATUS	PIPELINE	NOTES
<i>System or person</i>	<i>Digital / Organi</i>		<i>Clean / Needs W</i>	<i>Connected / Mani</i>	<i>AT RISK?</i>

- ☐ **Architect test.** Someone who wasn't in the room could read this and know what they're designing against.
- ☐ **One constraint, one page.** Every row touches the constraint. If it runs longer than a page, you've drifted into a data audit.
- ☐ **Layers, not just Records.** Knowledge and Semantics rows exist — or are explicitly named as gaps.
- ☐ **No blank pipelines, no empty Missing column.** Every digital row is Connected / Manual / Broken; the gaps are named.

Constraint + map. One page. **That's the input Design needs.**

The human inputs from *Knowledge That Walks* and the digital inputs from this worksheet are the same map seen from two sides. Put the Constraint Statement on top, the Knowledge Map below it, and hand both to Design. You can't architect a system against inputs you never made visible.